

Computational Sociology

Word embeddings and computational semantics

Dr. Thomas Davidson

University of North Carolina at Chapel Hill

September 25, 2026

Plan

1. Course updates
2. The vector-space model review
3. Latent semantic analysis
4. Language models
5. Word embeddings
6. Contextualized embeddings

Course updates

Project proposal

- ▶ Project proposals due Friday, October 2
 - ▶ Submit PDF via email
- ▶ See last week's slides for the proposal requirements

The vector-space model review

Vector representations

- ▶ Last week we looked at how we can represent texts as numeric vectors
 - ▶ Documents as vectors of words
 - ▶ Words as vectors of documents
- ▶ A document-term matrix (DTM) is a matrix where documents are represented as rows and tokens as columns

The vector-space model review

Weighting schemes

- ▶ We can use different schemes to weight these vectors
 - ▶ Binary (Does word w_i occur in document d_j ?)
 - ▶ Counts (How many times does word w_i occur in document d_j ?)
 - ▶ TF-IDF (How many times does word w_i occur in document d_j , accounting for how often w_i occurs across all documents $d \in D$?)
 - ▶ Recall *Zipf's Law*: a handful of words account for most words used; such words do little to help us to distinguish between documents

The vector-space model review

Cosine similarity

$$\cos(\theta) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|} = \frac{\sum_i \vec{u}_i \vec{v}_i}{\sqrt{\sum_i \vec{u}_i^2} \sqrt{\sum_i \vec{v}_i^2}}$$

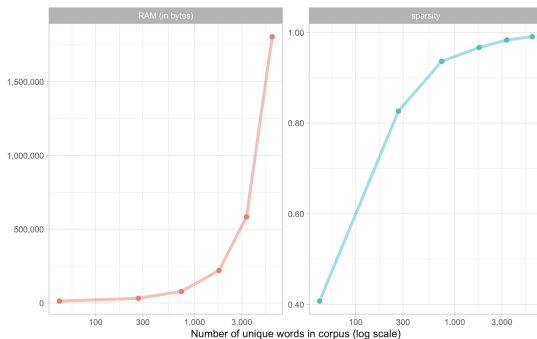
The vector-space model review

Limitations

- ▶ These methods produce *high-dimensional, sparse* vector representations
 - ▶ Given a vocabulary of unique tokens N the length of each vector $|V| = N$.
 - ▶ Many values will be zero since most documents only contain a small subset of the vocabulary.

The vector-space model review

Limitations



Source: <https://smltar.com/embeddings.html>

Latent semantic analysis

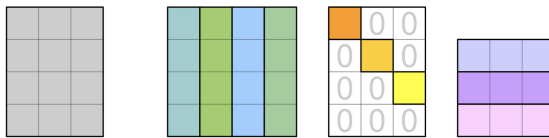
Latent Semantic Analysis

- ▶ One approach to reduce dimensionality and better capture semantics is called **Latent Semantic Analysis (LSA)**
 - ▶ We can use a process called *singular value decomposition* to find a *low-rank approximation* of a DTM.
- ▶ This provides *low-dimensional, dense* vector representations
 - ▶ Low-dimensional, since $k \ll |V|$
 - ▶ Dense, since vectors contain real values, with few zeros
- ▶ In short, we can “squash” a big matrix into a much smaller matrix while retaining important information.

$$DTM = U\Sigma V^T$$

Latent semantic analysis

Singular Value Decomposition



The diagram illustrates the Singular Value Decomposition (SVD) of a matrix M . It shows four matrices arranged horizontally, separated by an equals sign and a multiplication sign. The first matrix, M , is a 4x4 grid of gray squares. The second matrix, U , is a 4x4 grid with four columns of different colors: teal, green, blue, and light green. The third matrix, Σ , is a 4x4 grid with a diagonal of colored squares (orange, yellow, yellow, and light yellow) and zeros elsewhere. The fourth matrix, V^* , is a 4x4 grid with four rows of different colors: light blue, blue, purple, and pink.

$$\begin{matrix} \text{4x4} & & \text{4x4} & & \text{4x4} & & \text{4x4} \\ \mathbf{M} & = & \mathbf{U} & & \mathbf{\Sigma} & & \mathbf{V}^* \\ m \times n & & m \times m & & m \times n & & n \times n \end{matrix}$$

The Wikipedia page includes an animation of latent dimensions in a sparse DTM:
https://en.wikipedia.org/wiki/Latent_semantic_analysis

Latent semantic analysis

Loading data and packages

```
library(tidyverse)
library(tidytext)
library(ggplot2)
library(stringr)

df <- read_csv("../data/politics_twitter.csv")
dim(df)
```

```
## [1] 34220    13
```

```
length(unique(df$screen_name))
```

```
## [1] 12
```

Latent semantic analysis

Text preprocessing

```
texts.tidy <- df |>
  mutate(text = gsub("@\\w+", "", text)) |>
  mutate(text = gsub("#\\w+", "", text)) |>
  mutate(text = gsub(">", "'", text)) |>
  unnest_tokens(word, text) |>
  anti_join(stop_words) |>
  count(status_id, word) |>
  group_by(word) |>
  mutate(tweet_count = n_distinct(status_id),
         text_count = sum(n)) |>
  ungroup() |>
  filter(tweet_count >= 100 & text_count < 1E4) |>
  filter(!str_detect(word, "[0-9]+$")) |>
  bind_tf_idf(word, status_id, n)
```

Latent semantic analysis

Creating a DTM

```
X <- texts.tidy |>  
  cast_dtm(status_id, word, tf_idf) |>  
  as.matrix()
```

Latent semantic analysis

Creating a lookup dictionary

We can construct a list to allow us to easily find the index of a particular token.

```
lookup.index.from.token <- list()

for (i in 1:length(colnames(X))) {
  lookup.index.from.token[colnames(X)[i]] <- i
}
```

Latent semantic analysis

Using the lookup dictionary

This easily allows us to find the vector representation of a particular word. Note how most values are zero.

```
lookup.index.from.token["president"]
```

```
## $president  
## [1] 26
```

```
president.index <- unlist(lookup.index.from.token["president"])  
president.values <- round(as.numeric(X[, president.index]), 5)  
matrix(president.values[1:15], ncol = 5)
```

```
##      [,1]    [,2] [,3]    [,4]    [,5]  
## [1,]    0 0.00000    0 0.13748 0.00000  
## [2,]    0 0.31424    0 0.00000 0.00000  
## [3,]    0 0.00000    0 0.00000 0.54992
```

Latent semantic analysis

Calculating similarities

The following code normalizes each *column* and constructs a word-word cosine-similarity matrix.

```
normalize <- function(X) {  
  columnNorms <- sqrt(colSums(X^2))  
  Xn <- X / matrix(columnNorms,  
                    nrow = nrow(X),  
                    ncol = ncol(X), byrow = TRUE)  
  return(Xn)  
}  
  
X.n <- normalize(X)  
  
sims <- crossprod(X.n) # Optimized routine for t(X.n) %*% X.n  
dim(sims)  
  
## [1] 830 830
```

Latent semantic analysis

Most similar function

For a given token, this function allows us to find the n most similar tokens in the similarity matrix, where n defaults to 10.

```
get.top.n <- function(token, sims, n=10) {  
  top <- sort(sims[unlist(lookup.index.from.token[token]),],  
              decreasing=T)[1:n]  
  return(top)  
}
```

Latent semantic analysis

Finding similar words

```
get.top.n("", sims, n = 5)
```

Latent semantic analysis

Singular value decomposition

The `svd` function allows us to decompose the DTM. We can then easily reconstruct it using the formula shown above.

```
# Computing the singular value decomposition
lsa <- svd(X)

# We can recover the original matrix from this representation
X.2 <- lsa$u %*% diag(lsa$d) %*% t(lsa$v) # X = U \Sigma V^T

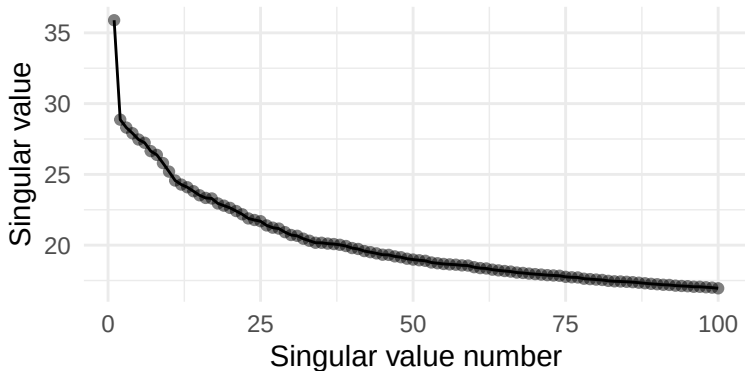
# Verifying that values are the same
max(abs(X - X.2))

## [1] 1.588073e-13
```

Latent semantic analysis

Singular value decomposition

First 100 singular values



Larger singular values account for more variation in the original matrix.

Latent semantic analysis

Truncated singular value decomposition

In the example above, we retained all the singular components. The point of latent semantic analysis is to compute a *truncated* SVD that retains only the first k components, giving us a rank- k approximation of X .

```
k <- 50 # Dimensions in truncated matrix

# Keep only the first k singular values
lsa.2 <- svd(X, nu=k, nv=k)

# Reconstruct X from those k values
X.trunc <- lsa.2$u %*% diag(lsa.2$d[1:k]) %*% t(lsa.2$v)

# The reconstruction is approximate
mean(abs(X-X.trunc))

## [1] 0.01041008
```

Latent semantic analysis

Inspecting the LSA matrix

```
words.lsa <- t(lsa.2$v)
colnames(words.lsa) <- colnames(X)

president.lsa <- words.lsa[, president.index]
president.lsa <- round(as.numeric(president.lsa), 3)
matrix(president.lsa[1:15], ncol = 5)

##           [,1]  [,2]  [,3]  [,4]  [,5]
## [1,] -0.316 -0.168  0.078 -0.140 -0.007
## [2,] -0.131  0.249 -0.097 -0.036 -0.019
## [3,] -0.190  0.236  0.588 -0.023  0.008
```

Latent semantic analysis

Recalculating similarities using the LSA matrix

```
words.lsa.n <- normalize(words.lsa)
sims.lsa <- t(words.lsa.n) %*% words.lsa.n
```

Latent semantic analysis

Comparing similarities

```
bind_cols(  
  original = names(get.top.n("economy", sims, n = 5)),  
  lsa = names(get.top.n("economy", sims.lsa, n = 5))  
)
```

```
## # A tibble: 5 x 2  
##   original lsa  
##   <chr>    <chr>  
## 1 economy economy  
## 2 build   paying  
## 3 rebuild invest  
## 4 wealth  economic  
## 5 growing rebuild
```

Latent semantic analysis

Comparing similarities

```
get.bottom.n <- function(token, sims, n=10) {  
  bottom <- sort(sims[unlist(lookup.index.from.token[token]),],  
                 decreasing=F)[1:n]  
  return(bottom)  
}  
  
get.bottom.n("health", sims.lsa, n = 5)
```

```
##      history      democrat      modern      georgia impeachment  
## -0.4301757 -0.3341455 -0.3315492 -0.2845085 -0.2816869
```

Latent semantic analysis

Comparing similarities

```
bind_cols(  
  original = names(get.top.n("health", sims, n = 5)),  
  lsa = names(get.top.n("health", sims.lsa, n = 5))  
)
```

```
## # A tibble: 5 x 2  
##   original    lsa  
##   <chr>      <chr>  
## 1 health    health  
## 2 care      care  
## 3 insurance insurance  
## 4 public    affordable  
## 5 affordable medicare
```

Latent semantic analysis

Exercise

Re-run the code above with a different value of k (try lower or higher). Compare some terms in the original similarity matrix and the new matrix. How does changing k affect the results?

```
get.top.n("", sims)
get.top.n("", sims.lsa)
```

Latent semantic analysis

Inspecting the latent dimensions

We can analyze the meaning of the latent dimensions by looking at the terms with the highest weights in each row. In this case I use the raw LSA matrix. What do you notice about the dimensions?

```
lsa_dimensions <- tibble(Word1 = character(),  
                          Word2 = character(),  
                          Word3 = character())  
  
for (i in 1:nrow(words.lsa)) {  
  top.words <- sort(words.lsa[i,], decreasing = TRUE)[1:3]  
  temp <- tibble(Word1 = names(top.words)[1],  
                 Word2 = names(top.words)[2],  
                 Word3 = names(top.words)[3])  
  lsa_dimensions <- bind_rows(lsa_dimensions, temp)  
}
```

Latent semantic analysis

Limitations of Latent Semantic Analysis

- ▶ Bag-of-words assumptions and document-level word associations
 - ▶ We still treat words as belonging to documents and lack finer context about their relationships
 - ▶ Although we could theoretically treat smaller units like sentences as documents
- ▶ Matrix computations become intractable with large corpora
- ▶ A neat linear algebra trick, but no underlying *language model*

Language models

Intuition

- ▶ A language model is a probabilistic model of language use
- ▶ Given some string of tokens, what is the most likely token?
 - ▶ Examples
 - ▶ Auto-complete
 - ▶ Google search completion

Language models

Bigram models

- ▶ $P(w_i|w_{i-1})$ = What is the probability of word w_i given the last word, w_{i-1} ?
 - ▶ $P(\textit{Carolina}|\textit{North})$
 - ▶ $P(\textit{Dakota}|\textit{North})$
 - ▶ $P(\textit{West}|\textit{North})$
 - ▶ $P(\textit{star}|\textit{North})$

Language models

Bigram models

- ▶ We use a corpus of text to calculate these probabilities from word co-occurrence.
 - ▶ $P(\textit{Carolina}|\textit{North}) = \frac{C(\textit{North Carolina})}{C(\textit{North})}$, i.e. proportion of times “North” is followed by “Carolina”, where $C()$ is the count operator.
- ▶ More frequently occurring pairs will have a higher probability.

Language models

Incorporating more information

- ▶ We can also model the probability of a word, given a sequence of words
- ▶ $P(x|S)$ = What is the probability of some word x given a partial sentence S ?
 - ▶ $A = P(\text{Hill} | \text{University of North Carolina at Chapel})$
 - ▶ $B = P(\text{Street} | \text{University of North Carolina at Chapel})$
 - ▶ $C = P(\text{Beach} | \text{University of North Carolina at Chapel})$
- ▶ The longer context makes “Hill” much more likely than the alternatives:
 - ▶ $A > B > C$.

Language models

Estimation

We can compute the probability of an entire sequence by considering the *conditional probability* of each token given the preceding tokens. For a sequence of n words, we want to know the joint probability $P(w_1, w_2, w_3, \dots, w_n)$. We can express this using the chain rule of probability:

$$\begin{aligned} P(w_{1:n}) &= P(w_1)P(w_2|w_1)P(w_3|w_{1:2})\dots P(w_n|w_{1:n-1}) \\ &= \prod_{k=1}^n P(w_k|w_{1:k-1}) \end{aligned}$$

Language models

Estimation

The bigram model simplifies this by assuming it is a first-order Markov process, such that the probability of w_k only depends on the previous word, w_{k-1} .

$$P(w_{1:n}) \approx \prod_{k=1}^n P(w_k | w_{k-1})$$

These probabilities can be estimated using maximum likelihood estimation on a corpus.

Language models

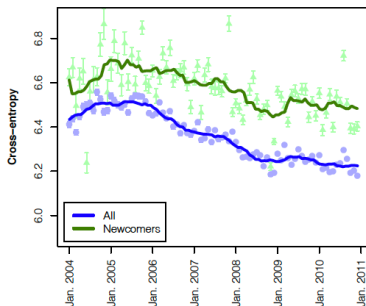
Empirical applications

- ▶ Danescu-Niculescu-Mizil et al. 2013 construct a bigram language model for each month on *BeerAdvocate* and *RateBeer* to capture the language of the community
 - ▶ For any given comment or user, they can then use a measure called *cross-entropy* to calculate how “surprising” the text is, given the assumptions about the language model
- ▶ The theory is that new users will take time to assimilate into the linguistic norms of the community

https://en.wikipedia.org/wiki/Cross_entropy

Language models

Empirical applications



(a) BeerAdvocate

Danescu-Niculescu-Mizil, Cristian, Robert West, Dan Jurafsky, Jure Leskovec, and Christopher Potts. 2013. "No Country for Old Members: User Lifecycle and Linguistic Change in Online Communities." In *Proceedings of the 22nd International Conference on World Wide Web*, 307–18. <https://doi.org/10.1145/2488388.2488416>

Language models

Limitations of N-gram language models

- ▶ Language use is much more complex than N-gram language models
- ▶ Three limitations:
 1. Insufficient for meaningful language generation
 2. More complex models become intractable to compute
 3. Fixed context windows cannot capture long-range dependencies

Language models

Neural language models

- ▶ The availability of large text corpora and advances in neural networks have enabled more expressive language models.
- ▶ By using machine learning to train a language model, we can construct better, more meaningful vector representations

Word embeddings

Intuition

- ▶ We use the context in which a word occurs to train a language model
 - ▶ The model learns by viewing millions of short snippets of text (e.g., 5-grams)
- ▶ This model outputs a vector representation of each word in k -dimensional space, where $k \ll |V|$.
 - ▶ Like LSA, these vectors are *dense*
 - ▶ Each element contains a real number and can be positive or negative

Word embeddings

Word2vec: Skip-gram and continuous bag-of-words (CBOW)

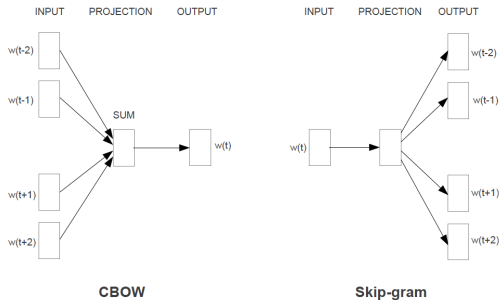


Figure 1: New model architectures. The CBOW architecture predicts the current word based on the context, and the Skip-gram predicts surrounding words given the current word.

Word embeddings

Word2vec: CBOW intuition

- ▶ We hide the focal word but give the model the words within a context window, in this case two words on either side
 - ▶ e.g. “The cat ? on the”, where ? = “sat”
- ▶ The model learns to assign a high probability to the observed focal word
- ▶ *Negative sampling* makes training efficient by contrasting the observed word with a small set of sampled alternatives
- ▶ Repeating this across millions of windows teaches the model which words occur in similar contexts

Word embeddings

Word2vec: Skip-gram intuition

- ▶ We give the model the focal word and ask it to predict words that occur nearby
 - ▶ e.g. “?₁ ?₂ sat ?₃ ?₄”
- ▶ Observed focal-word/context pairs are contrasted with sampled alternatives
- ▶ Repeating this across millions of windows teaches the model which focal words share contexts

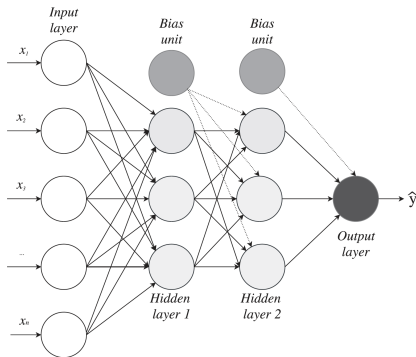
Word embeddings

Word2vec: Model

- ▶ Word2vec uses a shallow neural network to predict a word given a context (CBOW) or a context given a word (skip-gram)
 - ▶ But we do not care about the prediction itself, only the *weights* the model learns
- ▶ It is a self-supervised method since the model is able to update using the correct answers
 - ▶ e.g. In CBOW the model knows when the prediction is wrong and updates the weights accordingly

Word embeddings

Word2vec: Feed-forward neural network



This example shows a two-layer feed-forward neural network.

Word embeddings

Word2vec: Estimation procedure

- ▶ Batches of text are passed through the network
 - ▶ After each batch, weights are updated using *back-propagation*
 - ▶ The model updates its weights in the direction of the correct answer (the objective is to improve predictive accuracy)
 - ▶ Optimization via *stochastic gradient descent*

Word embeddings

Vector representations of words

- ▶ Each word is represented as a vector of weights learned by the neural network
 - ▶ Word embeddings are a *byproduct* of training a neural language model
 - ▶ Each element of this vector represents how strongly the word activates a neuron in the hidden layer of the network
 - ▶ This represents the association between the word and a given dimension in semantic space

Word embeddings

Distributional semantics

- ▶ The word vectors in the embedding space capture information about the context in which words are used
 - ▶ Words with similar meanings are situated close together in the embedding space
- ▶ *Distributional semantics* is the theory that the meaning of a word is derived from its context in language use
 - ▶ “You shall know a word by the company it keeps”, linguist J.R. Firth (1957)
- ▶ This is consistent with philosopher Ludwig Wittgenstein’s *use theory of meaning*
 - ▶ “the meaning of a word is its use in the language”, *Philosophical Investigations* (1953)

Word embeddings

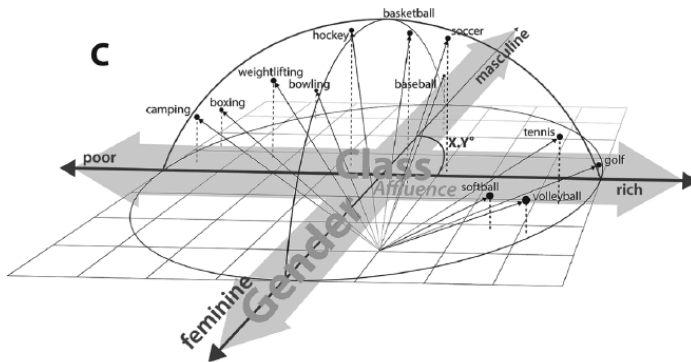
Analogies

- ▶ The most famous result from the initial word embedding paper is the ability of these vectors to capture analogies:
 - ▶ $\text{king} - \text{man} + \text{woman} \approx \text{queen}$
 - ▶ $\text{Madrid} - \text{Spain} + \text{France} \approx \text{Paris}$

Mikolov, Tomas, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. "Efficient Estimation of Word Representations in Vector Space." arXiv Preprint *arXiv:1301.3781* / Mikolov, Tomas, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean. 2013. "Distributed Representations of Words and Phrases and Their Compositionality." In *Advances in Neural Information Processing Systems*, 3111–19.

Word embeddings

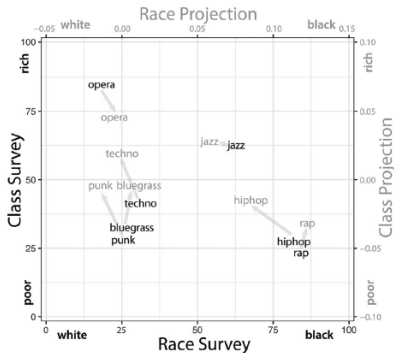
Sociological applications: Understanding social class



Kozlowski, Austin C., Matt Taddy, and James A. Evans. 2019. "The Geometry of Culture: Analyzing the Meanings of Class through Word Embeddings." *American Sociological Review* 84(5): 905–49.
<https://doi.org/10.1177/0003122419877135>

Word embeddings

Sociological applications: Understanding social class



Word embeddings

Context and concepts

Table 1 Context spaces versus concept spaces

Type of space	What it is	Entities within the space	Meaning of coordinates
Context space	Abstract many-dimensional space that summarizes the tendency of terms to be used together with other terms; the space that word embeddings directly estimate	Terms from the corpus (usually individual words or common phrases; sometimes word components like subwords, stems, etc.)	Coordinates are not themselves directly meaningful; two terms are proximate if they occur in similar usage contexts
Concept space	Abstract many-dimensional space that represents concepts in terms of their semantically meaningful features (like their size, status, or gender); the space social scientists are often interested in estimating	Abstract, language-independent concepts	Coordinates correspond to features people use to comprehend, discuss, and categorize concepts (e.g., gender, morality); feature dimensions can be objective or subjective, intrinsic or extrinsic, and universally accepted or subculturally specific. Two concepts are proximate if they have similar features.

Word embeddings

Sociological applications: Latent dimensions

Table 4

Term pairs for immigration-citizenship cultural dimension.

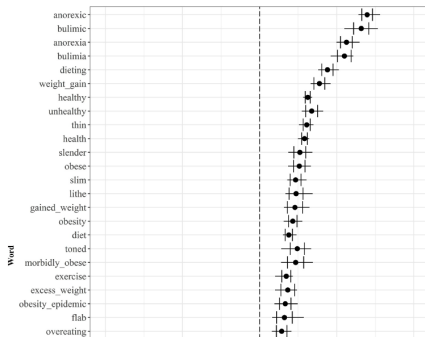
Immigrants	Citizens
Immigration	Citizenship
Immigrant	Citizen
Foreign	Domestic
Foreigner	Native
Outsider	Insider
Stranger	Local
Alien	Resident
Foreigner	Resident
Alien	Native
Immigrant	Local
Foreign	Familiar

Stoltz, Dustin S., and Marshall A. Taylor. 2021. "Cultural Cartography with Word Embeddings." *Poetics* 88 (October): 101567.

Word embeddings

Sociological applications: Understanding cultural schemas

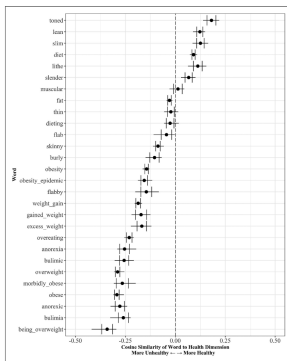
Figure 4: Gendering of Obesity-Related Words



Arseniev-Koehler, Alina, and Jacob G. Foster. 2022. "Machine Learning as a Model for Cultural Learning: Teaching an Algorithm What It Means to Be Fat." *Sociological Methods & Research* 51(4): 1484–1539.
<https://doi.org/10.1177/0049124119862488>

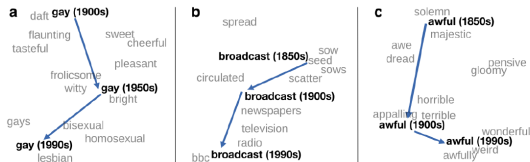
Word embeddings

Sociological applications: Understanding cultural schemas



Word embeddings

Sociological applications: Semantic change



Hamilton, William L., Jure Leskovec, and Dan Jurafsky. 2016. "Diachronic Word Embeddings Reveal Statistical Laws of Semantic Change." In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 1489–1501.

Word embeddings

Sociological applications: Semantic change

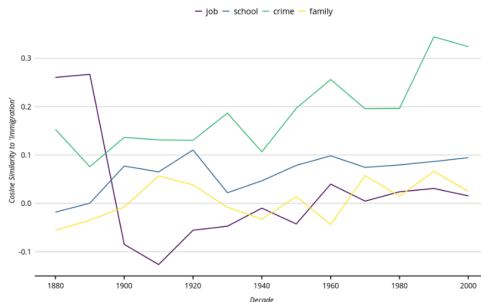
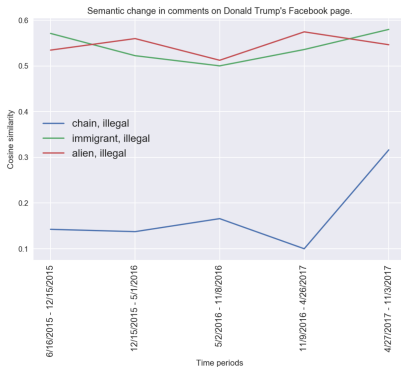


Fig. 1. Cosine Similarity of 'Immigration' and Key Terms by Decade, 1880 to 2000.

Stoltz, Dustin S., and Marshall A. Taylor. 2021. "Cultural Cartography with Word Embeddings." *Poetics* 88 (October): 101567.

Word embeddings

Sociological applications: Semantic change



Davidson 2017, unpublished.

Word embeddings

Sociological applications: Semantic change

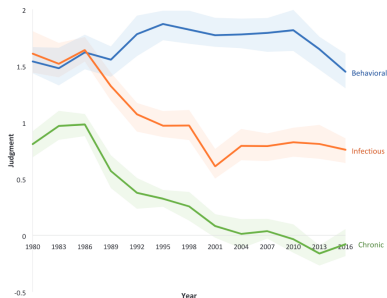


Figure 3. Judgment Scores for Behavioral, Infectious, and Chronic Diseases over Time
Note: More positive scores indicate stronger connotations of immorality and bad personality traits. More negative scores indicate stronger connotations of morality and good personality traits.

Best, Rachel Kahn, and Alina Arseniev-Koehler. 2023. "The Stigma of Diseases: Unequal Burden, Uneven Decline." *American Sociological Review* 88 (5): 938–69.

Word embeddings

Sociological applications: Embedding regression

Embedding Regression: Models for Context-Specific Description and Inference

PEDRO L. RODRIGUEZ *New York University, United States*

ARTHUR SPIRLING *New York University, United States*

BRANDON M. STEWART *Princeton University, United States*

Social scientists commonly seek to make statements about how word use varies over circumstances—including time, partisan identity, or some other document-level covariate. For example, researchers might wish to know how Republicans and Democrats diverge in their understanding of the term “immigration.” Building on the success of pretrained language models, we introduce the *à la carte* on text (conText) embedding regression model for this purpose. This fast and simple method produces valid vector representations of how words are used—and thus what words “mean”—in different contexts. We show that it outperforms slower, more complicated alternatives and works well even with very few documents. The model also allows for hypothesis testing and statements about statistical significance. We demonstrate that it can be used for a broad range of important tasks, including understanding US polarization, historical legislative development, and sentiment detection. We provide open-source software for fitting the model.

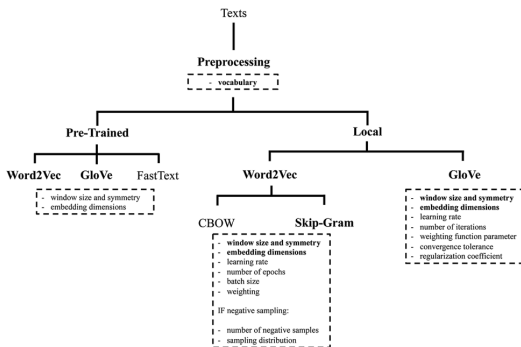
Word embeddings

Pre-trained word embeddings

- ▶ In addition to `word2vec` there are several other popular variants including `GloVe` and `fastText` (see Stoltz and Taylor 2021)
 - ▶ Pre-trained embeddings are available to download so you don't need to train your own
- ▶ When to train your own embeddings?
 - ▶ The language use in the target corpus differs from that represented by existing corpora
 - ▶ e.g. Arseniev-Koehler and Foster were interested in news coverage specific to health
 - ▶ Requires large numbers of documents ($> 10k$)

Word embeddings

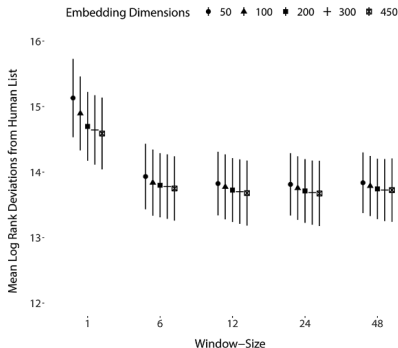
Selecting embeddings



Rodriguez, Pedro L., and Arthur Spirling. 2022. "Word Embeddings: What Works, What Doesn't, and How to Tell the Difference for Applied Research." *The Journal of Politics* 84(1): 101–15. <https://doi.org/10.1086/715162>

Word embeddings

Selecting embeddings



Rodriguez, Pedro L., and Arthur Spirling. 2022. "Word Embeddings: What Works, What Doesn't, and How to Tell the Difference for Applied Research." *The Journal of Politics* 84(1): 101–15. <https://doi.org/10.1086/715162>

Word embeddings

Selecting embeddings

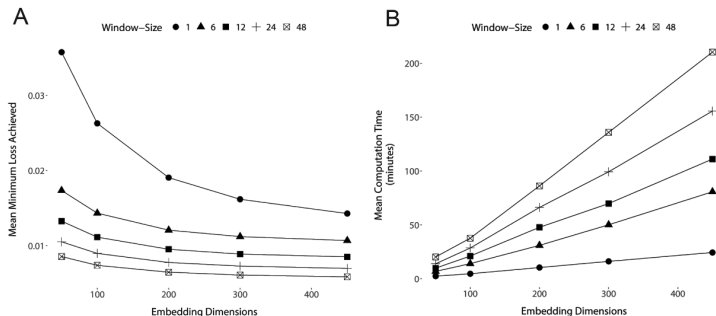


Figure 4. Technical criteria: larger models fit better but take longer to compute. A, Mean minimum loss achieved. B, Computation time (minutes)

Rodriguez, Pedro L., and Arthur Spirling. 2022. "Word Embeddings: What Works, What Doesn't, and How to Tell the Difference for Applied Research." *The Journal of Politics* 84(1): 101–15. <https://doi.org/10.1086/715162>

Word embeddings

Word embeddings in R

We will use word2vec, an R wrapper around a C++ library.

Original library: <https://github.com/maxoodf/word2vec>

R package: <https://github.com/bnosac/word2vec>

```
#install.packages("word2vec")  
library(word2vec)  
set.seed(8901) # random seed
```

Word embeddings

Training a word2vec model

```
model <- word2vec::word2vec(  
  x = tolower(df$text),  
  type = "cbow",  
  dim = 100,  
  window = 3L,  
  iter = 10L,  
  negative = 10L,  
  min_count = 10L  
)
```

Word embeddings

Getting embeddings for words

We can use the `predict` function to find the nearest words to a given term.

```
predict(model, c("border"), type = "nearest", top_n = 10)
```

```
## $border
##   term1      term2 similarity rank
## 1 border immigration 0.7383195    1
## 2 border   borders 0.6913826    2
## 3 border  southern 0.6897542    3
## 4 border     mmiw 0.6569045    4
## 5 border   migrant 0.6555419    5
## 6 border   eastern 0.6548232    6
## 7 border  portland 0.6263564    7
## 8 border humanitarian 0.6219962    8
## 9 border    climate 0.6204214    9
## 10 border  forefront 0.6106523   10
```

Word embeddings

Testing analogical reasoning

```
emb <- as.matrix(model) # Extracting embedding matrix  
vector <- emb["king", ] - emb["man", ] + emb["woman", ]  
predict(model, vector, type = "nearest", top_n = 10)
```

##	term	similarity	rank
## 1	king	0.9638355	1
## 2	martin	0.8660259	2
## 3	luther	0.8475783	3
## 4	jr	0.8386757	4
## 5	spaceforcedod	0.8196986	5
## 6	michael	0.8163184	6
## 7	rbg	0.8109796	7
## 8	james	0.8068820	8
## 9	marched	0.7957851	9
## 10	damian	0.7937300	10

Word embeddings

Testing analogical reasoning

```
vector <- emb["austin", ] - emb["texas", ] + emb["california", ]  
predict(model, vector, type = "nearest", top_n = 10)
```

##	term	similarity	rank
## 1	austin	0.9918814	1
## 2	lloyd	0.9243966	2
## 3	milley	0.8864709	3
## 4	haaland	0.8752555	4
## 5	california's	0.8746035	5
## 6	inspector	0.8727241	6
## 7	dejoy	0.8714325	7
## 8	postmaster	0.8679774	8
## 9	california	0.8649999	9
## 10	attorney	0.8578929	10

Word embeddings

Loading a pre-trained embedding

We will use 50-dimensional GloVe vectors trained on six billion tokens from Wikipedia and Gigaword. The model contains vectors for 400,000 terms and is 163 MB uncompressed.

Model source: <https://github.com/piskvorky/gensim-data>

Package documentation: <https://github.com/bnosac/word2vec>

```
glove <- read.wordvectors(  
  file = "../data/glove-wiki-gigaword-50",  
  type = "txt", normalize = TRUE  
)  
  
nearest <- function(x, n = 10) {  
  if (is.character(x)) x <- glove[x, , drop = FALSE]  
  if (is.vector(x)) {  
    x <- matrix(x, nrow = 1, dimnames = list("query", NULL))  
  }  
  word2vec_similarity(x, glove, top_n = n, type = "cosine")  
}
```

Word embeddings

Similarities

Find the top 10 most similar terms to “economy” in the embedding space.

```
nearest("economy")
```

##		term1	term2	similarity	rank
## 1	economy	economy	1.0000000	1	
## 2	economy	economic	0.8861514	2	
## 3	economy	growth	0.8313134	3	
## 4	economy	recession	0.8276588	4	
## 5	economy	economies	0.8157053	5	
## 6	economy	sector	0.8131152	6	
## 7	economy	slowing	0.8090310	7	
## 8	economy	recovery	0.8055825	8	
## 9	economy	market	0.8026539	9	
## 10	economy	downturn	0.7988218	10	

Word embeddings

Similarities

Find the top 10 most similar terms to “immigration” in the embedding space.

```
nearest("immigration")
```

##		term1	term2	similarity	rank
## 1	immigration	immigration	1.0000000	1	
## 2	immigration	laws	0.7916791	2	
## 3	immigration	enforcement	0.7875627	3	
## 4	immigration	immigrants	0.7648074	4	
## 5	immigration	federal	0.7609670	5	
## 6	immigration	law	0.7607204	6	
## 7	immigration	legislation	0.7576876	7	
## 8	immigration	restrictions	0.7517299	8	
## 9	immigration	policy	0.7426129	9	
## 10	immigration	legal	0.7230684	10	

Word embeddings

Repeating the analogy test

Let's retry the analogy test. queen is the closest new term after king itself.

```
vector <- glove["king", ] - glove["man", ] + glove["woman", ]  
nearest(vector)
```

##		term1	term2	similarity	rank
## 1	query	king	0.8854023	1	
## 2	query	queen	0.8523603	2	
## 3	query	throne	0.7664334	3	
## 4	query	prince	0.7592144	4	
## 5	query	daughter	0.7473882	5	
## 6	query	elizabeth	0.7460220	6	
## 7	query	princess	0.7424571	7	
## 8	query	kingdom	0.7337412	8	
## 9	query	monarch	0.7214491	9	
## 10	query	eldest	0.7184862	10	

Word embeddings

Repeating the analogy test

The model also recovers Paris from the capital-country analogy.

```
vector <- glove["madrid", ] - glove["spain", ] + glove["france", ]  
nearest(vector)
```

##		term1	term2	similarity	rank
## 1	query	paris	0.8246473	1	
## 2	query	marseille	0.8051085	2	
## 3	query	madrid	0.8033524	3	
## 4	query	lyon	0.8015593	4	
## 5	query	france	0.7544605	5	
## 6	query	milan	0.7369778	6	
## 7	query	barcelona	0.7363471	7	
## 8	query	munich	0.7298524	8	
## 9	query	strasbourg	0.7143057	9	
## 10	query	lille	0.7125369	10	

Word embeddings

Representing documents

Last week we focused on how we could represent documents using the rows in the DTM. So far we have just considered how words are represented in the embedding space. We can represent a document by averaging over its composite words.

```
tokens <- c("university", "of", "north", "carolina", "at", "chapel", "hill")  
sentence <- colMeans(glove[tokens, ])
```

Word embeddings

Representing sentences and documents

- ▶ Averaging word vectors is a crude approximation that ignores word order and context
- ▶ Dedicated models such as Sentence-BERT can produce embeddings for sentences or short documents

Word embeddings

Visualizing high-dimensional embeddings in low-dimensional space

- ▶ There are various algorithms available for visualizing word embeddings in low-dimensional space
 - ▶ PCA, t-SNE, UMAP
- ▶ There are also browser-based interactive embedding explorers
 - ▶ TensorFlow Embedding Projector:
<https://projector.tensorflow.org/>

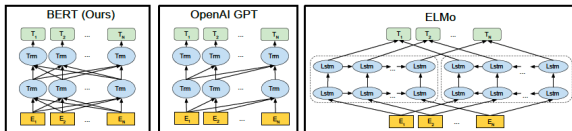
Contextualized embeddings

Limitations of existing approaches

- ▶ Word2vec and other embedding methods run into problems when dealing with *polysemy*
 - ▶ e.g. The vector for “crane” will be learned by averaging across different uses of the term: bird, construction equipment, movement
 - ▶ “She had to crane her neck to see the crane perched on top of the crane.”
- ▶ New methods have been developed to allow the vector for “crane” to vary according to different contexts
 - ▶ Intuition: Meaning varies depending on context
 - ▶ e.g. Proximity to “neck” implies “crane” used to describe movement

Contextualized embeddings

Architectures



Source: Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. "BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding." In *Proceedings of NAACL-HLT 2019*, 4171–86. ACL.

Contextualized embeddings

Methodological innovations

- ▶ Deep neural networks
- ▶ Pretraining objectives such as predicting masked tokens
- ▶ Subword tokenization, which can represent rare and previously unseen words
- ▶ Much more data and enormous compute power required
 - ▶ e.g. BERT was trained on a 3.3-billion-word corpus for 40 epochs using 64 TPU chips

Summary

- ▶ Limitations of sparse representations of text
 - ▶ LSA allows us to project sparse matrix into a dense, low-dimensional representation
- ▶ Probabilistic language models allow us to directly model language use
- ▶ Word embeddings use a neural language model to represent texts as dense vectors
 - ▶ Distributional semantics
- ▶ Recent methodological advances better incorporate context

Next week

- ▶ Topic modeling